STRUCTS & ENUMS

Scalar Types

Integers, float, bool, char

Compound Types

Tuples, Arrays

Function Types

Functions

Collection Types

Vec, String, Hashmap, …

Custom Types

Enum, struct

Special Types

Unit types, &T, &mut T

Smart pointers

Box, Rc, Arc, RefCell, …

Custom types

Category	Types
Struct	`struct Point { ... }`
Enum	`enum Direction { ... }`

I. STRUCTS

Definition

A **struct** is a custom data type used to group together related values under one name. Each value inside a struct is a **field**, and each field can have its own type.

1) Create a struct

```
struct Person {
    name: String,
    age: u8,
}
```

2) Create an object

```
let student = Person {
    name: String::from("Alice"),
    age: 20,
};
```

3) Call the object

```
println!("Name: {}", student.name);
println!("Age: {}", student.age);
```

Reminder

```rust
struct Person {
    name: String,
    age: u8,
}
```

1) Create a struct

```rust
let student = Person {
    name: String::from("Alice"),
    age: 20,
};
```

2) Create the object

```rust
println!("Name: {}", student.name);
println!("Age: {}", student.age);
```

3) Call the object

Exercice 1:
Struct basics

Default pattern

```rust
#[derive(Default)]
struct Person {
    name: String,
    age: u8,
}

fn main() {
    let p = Person::default(); // uses default values
    println!("Name: {}, Age: {}", p.name, p.age);
}
```

Exercice 2:
The `Default` pattern

The `new` pattern

✅ Definition: `new` Pattern in Rust

In Rust, the `new` **pattern** is a common convention where you define an **associated function named** `new()` to create and return an instance of a struct or enum.

🧠 Why use the `new` pattern?

- To make creating values **clean and consistent**
- To **hide internal details** (like using `String::from`)
- To allow for more logic later (e.g. validation, defaults)

The new pattern : example

```
struct Person {
    name: String,
}

impl Person {
    fn new(name: &str) -> Self {
        Person {
            name: name.to_string(),
        }
    }
}



fn main() {
    let p = Person::new("Alice");
}
```

Using self in structs

```rust
struct Person {
    name: String,
}

impl Person {
    // A method that uses &self to access the name
    fn say_hello(&self) {
        println!("Hello, my name is {}", self.name);
    }
}

fn main() {
    let person = Person {
        name: String::from("Alice"),
    };

    person.say_hello(); // Output: Hello, my name is Alice
}
```

Exercice 3:
The `new` pattern

You can also have tuples in struct.. !

```
struct Color(u8, u8, u8);
```

Tuple vs struct

Feature	Tuple	Struct
Field Access	By index (`.0` , `.1` , ...)	By name (`.name` , `.age`)
Field Order	**Matters**	**Doesn't matter**
Readability	Low	High
Use Case	Small, temporary data	Named, structured data

Exercice 4:
Tuples in struct

By default, structs have **private fields**!

Exercice 5:
Private fields

II. ENUMS

Definition

An **enum** is a custom data type that allows you to define a **type by listing all possible values** it can have.

“This value can be one of a **few specific** choices.”

Example

```rust
enum LightState {
    On,
    Off,
}

fn print_state(state: LightState) {
    match state {
        LightState::On => println!("The light is ON."),
        LightState::Off => println!("The light is OFF."),
    }
}

fn main() {
    let current = LightState::On;
    print_state(current);
}
```

Exercice 6:
Enum Directions

Enum inside struct

```rust
enum Role {
    Student,
    Teacher,
}

struct Person {
    name: String,
    role: Role,
}

fn describe(person: &Person) {
    match person.role {
        Role::Student => println!("{} is a student.", person.name),
        Role::Teacher => println!("{} is a teacher.", person.name),
    }
}
```

Exercice 7:
Enum inside struct

Your turn!